What is machine learning? Train/test split

🎯 Today's goal

By the end of this hour you will:

- know the difference between writing **rules** and **learning from examples**
- split a table into **features** (`X`) and a **label** (`y`)
- split the data into a **training set** and a **test set**, and know why
- train your first model with scikit-learn: **create** → **fit** → **predict** → **score**
- compare your model with a simple **baseline**

🤔 Why this matters

Until now, *you* found the patterns: on Day 15 you discovered that women and first-class passengers survived more often. **Machine learning** (ML) means letting the computer find patterns by itself, from examples, and then use them to make guesses about new cases it has never seen.

It's how a child learns what a dog is. Nobody gives them a rule book ("four legs, fur, barks..."). They see many dogs and many cats, someone says which is which, and after a while they can recognise a dog they've never met. Spam filters, film recommendations and medical image tools all learn this way.

📏 Rules versus learning

The old way is to write the rules yourself with `if` / `else` (Day 4). Here's a hand-written rule that guesses a fruit from its weight and how smooth its skin is (0 = very bumpy, 10 = very smooth):

```
def guess_fruit(weight, smoothness):
    """A rule I wrote myself after looking at some fruit."""
    if smoothness > 6:
        return "apple"
    else:
        return "orange"

print(guess_fruit(150, 8))
print(guess_fruit(170, 3))
print(guess_fruit(140, 6))   # smooth-ish orange or bumpy apple?
```

```
apple
orange
orange
```

This works for easy cases, but who decided that `6` is the right cut-off? And what about the weight? With 2 measurements you can still invent rules; with 30 measurements, nobody can. In machine learning we turn it around:

- **Rules:** you write the rules → the computer applies them to data → answers.
- **Machine learning:** you give data *and* the right answers → the computer *works out* the rules → it can answer new cases.

📖 The words you'll hear every day from now on

Word	Meaning	In the fruit example
feature	a measurement we know about each example (an input column)	weight, smoothness
label (or **target**)	the answer we want to predict	"apple" or "orange"
model	the thing that learns the pattern and then makes guesses	a "fruit guesser"
fit (or **train**)	let the model learn from examples with known labels	show it 100 labelled fruits
predict	ask the model for its guess on new examples	"what is this new fruit?"

By tradition the features are stored in a variable called `X` (capital, because it's a whole table) and the labels in `y` (lower case, because it's one column).

Two more pairs of words describe the *kind* of problem:

- **Supervised** learning: every example comes with its label, like a teacher who gives the answers while you practise. That's all of this week. **Unsupervised** learning: no labels at all, the computer just looks for groups (Day 21).
- **Classification**: the label is a category (apple / orange, survived / didn't, spam / not spam). **Regression**: the label is a number (a price, a temperature). Day 17 is about regression.

🌸 Our first real dataset: iris flowers

scikit-learn (imported as `sklearn`) is the most popular Python library for machine learning. It comes with a few small datasets built in. The most famous one measures 150 *iris* flowers of three species. This one is **real** data, collected in the 1930s, and it's been used to teach ML for decades:

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)     # as_frame=True gives us pandas tables
flowers = iris.frame
flowers.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Four measurements in centimetres (the *sepal* is the green leaf under the petals), and a `target` column with the species as a number. The numbers stand for names:

```
print("Species names:", iris.target_names.tolist())   # 0, 1, 2 in this order
print()
print(flowers["target"].value_counts())
```

```
Species names: ['setosa', 'versicolor', 'virginica']

target
0    50
1    50
2    50
Name: count, dtype: int64
```

So `0` = setosa, `1` = versicolor, `2` = virginica, with 50 flowers each. Models need numbers, which is why the species are stored as 0, 1, 2.

✂️ Features `X` and label `y`

The features are the four measurement columns; the label is `target`. You could pick them with pandas (Day 10), but `iris` already has them ready:

```
X = iris.data       # the 4 measurement columns (a DataFrame)
y = iris.target     # the species, 0 / 1 / 2 (a Series)

print("X shape:", X.shape)   # 150 rows, 4 features
print("y shape:", y.shape)   # 150 labels
X.head(3)
```

```
X shape: (150, 4)
y shape: (150,)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2

📚 Why we hide some data: the train/test split

Imagine a teacher gives the class the exact exam questions to practise. Everybody scores 100%, but did anybody *learn* anything, or did they just **memorise**? The only fair test uses questions the students have never seen.

Models are the same. So before training, we put part of the data aside:

- the **training set** (usually 70–80%): the model learns from it
- the **test set** (the other 20–30%): kept hidden, used only at the end to check how well the model does on flowers it has never seen

`train_test_split` shuffles the rows and cuts them into the two parts:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,      # 25% of the rows go to the test set
    random_state=42,     # shuffle the same way every time
    stratify=y,          # keep the same mix of species in both parts
)

print("Training rows:", len(X_train))
print("Test rows:    ", len(X_test))
```

```
Training rows: 112
Test rows:     38
```

It returns **four** things, always in this order: `X_train, X_test, y_train, y_test`. You unpack them into four variables in one line, like you unpacked tuples on Day 6.

`random_state=42` works like the NumPy seed: the shuffle is "random" but the same every time, so your results match this page. `stratify=y` makes sure each species is shared out fairly. Without it, by bad luck, the test set might get almost no setosas:

```
print("Species in the test set:")
print(y_test.value_counts())
```

```
Species in the test set:
target
1    13
2    13
0    12
Name: count, dtype: int64
```

🤖 Your first model

Every scikit-learn model is used with the same four steps. Learn them once and you can use dozens of different models:

1. **create** the model: `model = SomeModel()`
2. **fit** it on the training data: `model.fit(X_train, y_train)`
3. **predict** new cases: `model.predict(X_test)`
4. **score** it on the test data: `model.score(X_test, y_test)`

Our first model is called **k-nearest neighbours** (KNN). Its idea is simple: to guess the species of a new flower, find the 5 training flowers that are most similar to it, and let them vote. You'll look inside it on Day 18.

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)   # 1. create
model.fit(X_train, y_train)                   # 2. fit (learn)

predictions = model.predict(X_test)           # 3. predict the test set
print("Model's guesses:", predictions[:10].tolist())
print("Real answers:   ", y_test.head(10).tolist())
```

```
Model's guesses: [0, 1, 1, 1, 0, 1, 2, 2, 2, 2]
Real answers:    [0, 1, 1, 1, 0, 1, 2, 2, 2, 2]
```

The first 10 guesses look right. To measure all of them, use **accuracy**: the share of test examples the model got right. `score` calculates it:

```
accuracy = model.score(X_test, y_test)        # 4. score
print(f"Accuracy on unseen flowers: {accuracy:.1%}")
```

```
Accuracy on unseen flowers: 97.4%
```

There's no magic inside `score`. Compare the guesses with the answers (that gives True/False for each flower) and take the mean, the same 0/1 trick as yesterday's survival rates:

```
correct = predictions == y_test        # True where the guess was right
print("Right:", correct.sum(), "of", len(y_test))
print("Accuracy by hand:", correct.mean())
```

```
Right: 37 of 38
Accuracy by hand: 0.9736842105263158
```

A model must be fitted before it can predict. A brand-new model hasn't learned anything yet, so asking it for a guess gives an error:

💡 This example has a mistake on purpose. Run it and read the error.

```
new_model = KNeighborsClassifier()
new_model.predict(X_test)        # oops: we forgot new_model.fit(...)
```

⚠️ **NotFittedError**

```
Traceback (most recent call last):
  line 2, in <module>
    new_model.predict(X_test)        # oops: we forgot new_model.fit(...)
    ~~~~~~~~~~~~~~~~~^^^^^^^^
sklearn.exceptions.NotFittedError: This KNeighborsClassifier instance is not
fitted yet. Call 'fit' with appropriate arguments before using this estimator.
```

The message says it plainly: *not fitted yet. Call 'fit' … before using this estimator* ("estimator" is scikit-learn's word for a model).

🔮 Predicting brand-new flowers

The whole point of a model is to answer questions about new cases. Here are two flowers we just "measured" (made-up measurements). They must be in the same 4 columns as `X`, so we put them in a small DataFrame:

```
new_flowers = pd.DataFrame(
    [[5.0, 3.4, 1.5, 0.2],      # short, narrow petals
     [6.5, 3.0, 5.5, 2.0]],     # long, wide petals
    columns=X.columns,
)
guesses = model.predict(new_flowers)
for number in guesses:
    print("This looks like a", iris.target_names[number])
```

```
This looks like a setosa
This looks like a virginica
```

Two pairs of brackets: models always expect a *table* of examples, even for a single flower. One flower is one row: `[[5.0, 3.4, 1.5, 0.2]]`.

🧠 Memorising versus learning

Here's the exam story in code. A **decision tree** (Day 19) is a model that can memorise its training data perfectly. Let's score it on the training set *and* on the test set:

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)

print(f"Score on the training set: {tree.score(X_train, y_train):.1%}")
print(f"Score on the test set:     {tree.score(X_test, y_test):.1%}")
```

```
Score on the training set: 100.0%
Score on the test set:     89.5%
```

100% on the flowers it practised on, but less on new ones. If we had only checked the training set, we'd believe the model was perfect. **Always judge a model on data it hasn't seen.**

🪜 Is 97% good? Compare with a baseline

A score means little on its own. A **baseline** is the score of a "lazy" model that doesn't look at the features at all, for example one that always guesses the most common species. A real model must beat it clearly, otherwise it learned nothing useful. scikit-learn has such a lazy model, called `DummyClassifier`:

```
from sklearn.dummy import DummyClassifier

# always guesses the most common label:
lazy = DummyClassifier(strategy="most_frequent")
lazy.fit(X_train, y_train)

print(f"Baseline accuracy: {lazy.score(X_test, y_test):.1%}")
print(f"KNN accuracy:      {model.score(X_test, y_test):.1%}")
```

```
Baseline accuracy: 31.6%
KNN accuracy:      97.4%
```

With three equal species, always guessing the same one is right about a third of the time. Our model is far better than that. Baselines matter even more when one answer is very common: on Day 15's ship, guessing "did not survive" for everyone would already be right 59% of the time.

⚠️ Common mistakes

- Judging a model on its training data. It may have just memorised it.
- Mixing up the order: it's `X_train, X_test, y_train, y_test = train_test_split(X, y, ...)`.
- Calling `predict` before `fit` → `NotFittedError`.

- Giving one example with single brackets `[5.0, 3.4, 1.5, 0.2]`: models want a table, `[[...]]`.
- Forgetting `random_state=42`, so your numbers change every run and don't match the page.
- Getting excited about an accuracy without checking the baseline.

✅ Recap

- Machine learning = learning the rules from labelled examples, instead of writing them by hand.
- Features `X` are the inputs, the label `y` is the answer. Classification predicts categories, regression predicts numbers.
- `train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)` hides a test set.
- Every model: **create** → **fit** → **predict** → **score**.
- Accuracy = share of correct guesses = `(predictions == y_test).mean()`.
- Always compare with a baseline such as `DummyClassifier(strategy="most_frequent")`.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Features and label

A made-up table of fruits. Store the two feature columns `weight` and `smoothness` in `X` (a DataFrame), and the `fruit` column in `y` (the label).

```
import pandas as pd

fruits = pd.DataFrame({
    "weight": [150, 170, 140, 130, 180, 160],
    "smoothness": [8, 3, 7, 9, 2, 4],
    "fruit": ["apple", "orange", "apple", "apple", "orange", "orange"],
})

# X = ...
# y = ...
```

Hints:

1. Several columns: a list inside the square brackets, `fruits[["weight", "smoothness"]]`.
2. One column: `fruits["fruit"]`.

2. Split the flowers

Split the iris data into `X_train, X_test, y_train, y_test` with **20%** of the rows in the test set, `random_state=42` and a fair mix of species (`stratify`). Then print how many rows each part has.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris(as_frame=True)
X = iris.data
y = iris.target

# Split here
```

Hints:

1. 20% is `test_size=0.2`.
2. The order of the four results is `X_train, X_test, y_train, y_test`.
3. `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`

3. Train your own model

The data is already split. Create a `KNeighborsClassifier` called `model`, fit it on the **training** set, and store its accuracy on the **test** set in `accuracy`. It should reach at least 90%.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42, stratify=iris.target)

# Create, fit and score the model
```

Hints:

1. Create: `model = KNeighborsClassifier(n_neighbors=5)`
2. Fit on the training part: `model.fit(X_train, y_train)`
3. Score on the test part: `accuracy = model.score(X_test, y_test)`

4. Accuracy by hand

A made-up spam filter guessed 10 e-mails (1 = spam, 0 = not spam). Without scikit-learn, calculate the share of correct guesses and store it in `accuracy`.

```
import numpy as np

real_answers = np.array([1, 0, 0, 1, 1, 0, 0, 0, 1, 0])
guesses      = np.array([1, 0, 1, 1, 0, 0, 0, 0, 1, 0])

# accuracy = ...
```

Hints:

1. `guesses == real_answers` gives True where a guess is right.
2. The mean of True/False values is the share of Trues.
3. `accuracy = (guesses == real_answers).mean()`

5. Challenge: baseline versus model

Using the split in the starter, store the test accuracy of a `DummyClassifier(strategy="most_frequent")` in `baseline_acc` and of a `KNeighborsClassifier(n_neighbors=5)` in `model_acc`. Then store how many percentage points better the model is in `gain` (for example, 0.95 – 0.40 = 0.55 is a gain of `55.0`), and print a line like `The model beats the baseline by 55.0 points`.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.dummy import DummyClassifier

iris = load_iris(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42, stratify=iris.target)

# baseline_acc = ...
# model_acc = ...
# gain = ...
```

Hints:

1. Both models follow the same steps: create, `fit(X_train, y_train)`, `score(X_test, y_test)`.
2. Percentage points: `gain = (model_acc - baseline_acc) * 100`
3. `print(f"The model beats the baseline by {gain:.1f} points")`

➡️ Tomorrow

Linear regression: predicting numbers. Today the label was a category (a species). Tomorrow it's a number: you'll predict made-up apartment prices from their size by fitting a straight line, learn how to measure how far off the predictions are (MAE, RMSE, R^2), and then use several features at once.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Features and label

```
import pandas as pd

fruits = pd.DataFrame({
    "weight": [150, 170, 140, 130, 180, 160],
    "smoothness": [8, 3, 7, 9, 2, 4],
    "fruit": ["apple", "orange", "apple", "apple", "orange", "orange"],
})

X = fruits[["weight", "smoothness"]]
y = fruits["fruit"]
```

2. Split the flowers

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris(as_frame=True)
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Training rows:", len(X_train))
print("Test rows:", len(X_test))
```

3. Train your own model

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42, stratify=iris.target)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
print(f"Accuracy: {accuracy:.1%}")
```

4. Accuracy by hand

```
import numpy as np

real_answers = np.array([1, 0, 0, 1, 1, 0, 0, 0, 1, 0])
guesses      = np.array([1, 0, 1, 1, 0, 0, 0, 0, 1, 0])

accuracy = (guesses == real_answers).mean()
print(accuracy)
```

5. Challenge: baseline versus model

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.dummy import DummyClassifier

iris = load_iris(as_frame=True)
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.3, random_state=42, stratify=iris.target)

lazy = DummyClassifier(strategy="most_frequent")
lazy.fit(X_train, y_train)
baseline_acc = lazy.score(X_test, y_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
model_acc = model.score(X_test, y_test)

gain = (model_acc - baseline_acc) * 100
print(f"The model beats the baseline by {gain:.1f} points")
```